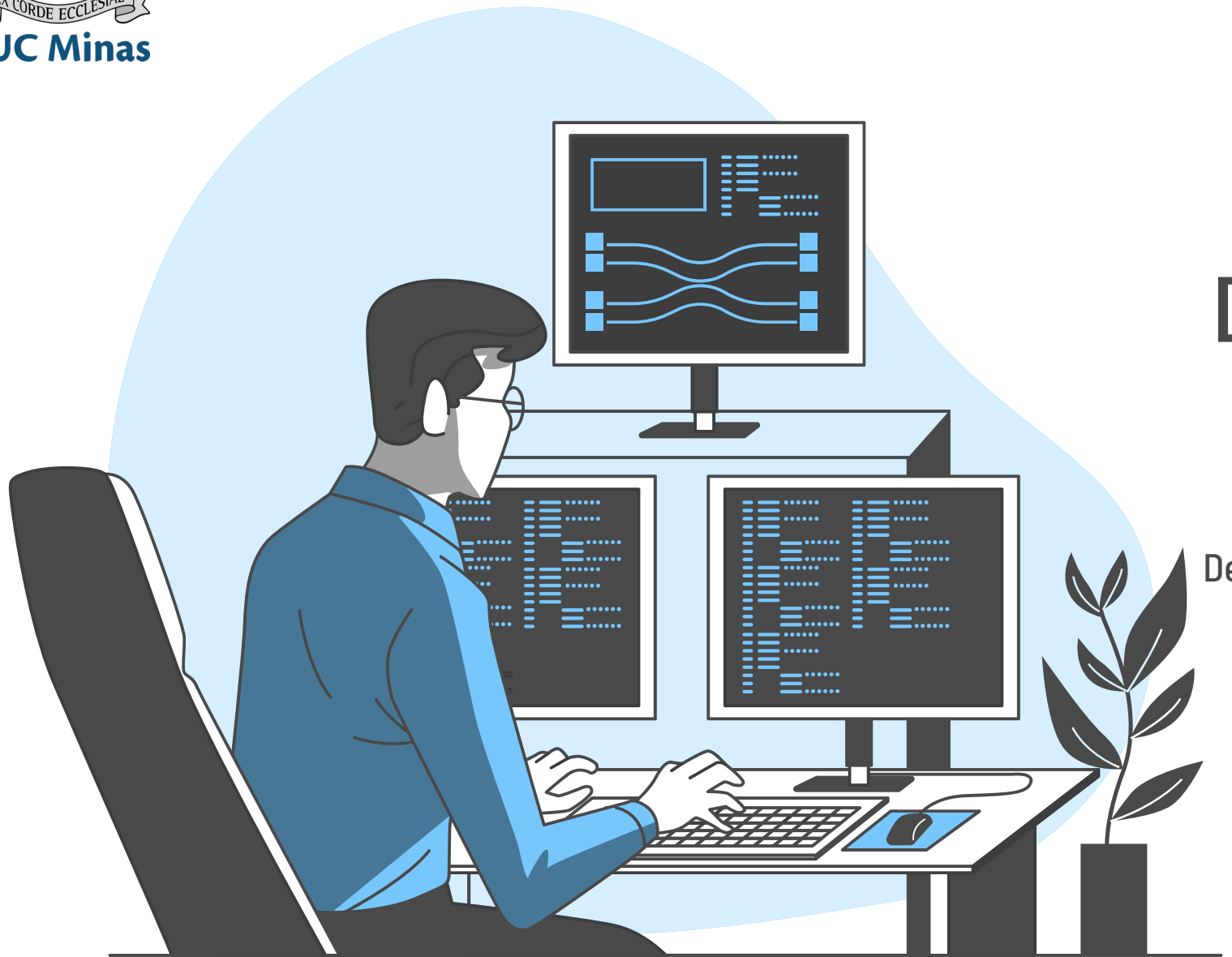
PUC Minas



Aula 05 · Deploy

Deploy do React + Vite no Vercel

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Importar um repositório do GitHub no **Vercel**

2

Configurar **Root Directory**, *Framework Preset*, build e pasta de saída

3

Cadastrar **variáveis de ambiente** `VITE_*`

4

Corrigir o 404 em rotas com `vercel.json`

5

Entender **deploys de produção e de preview** e quando fazer **redeploy**

Antes de começar

- Projeto React + Vite no GitHub
- `npm run build` funcionando localmente
- Conta no Vercel (login com GitHub)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Antes de começar

02 Passo 1: importar o repositório

03 Passo 2: configurar o projeto

04 Passo 3: acompanhar o build

05 Passo 4: site no ar

06 Passo 5: deploy contínuo e previews

07 Passo 6: vercel.json para rotas de SPA

08 Editando variáveis depois

09 Erros comuns no Vercel

10 Aprofundando: Vercel além do básico

01

Antes de começar

Antes de começar

Confirme que `npm run build` passa no seu computador e que o código está no GitHub. Se o projeto tem rotas (React Router), já crie o `vercel.json` (passo 6) antes do primeiro deploy.

01

Antes de começar

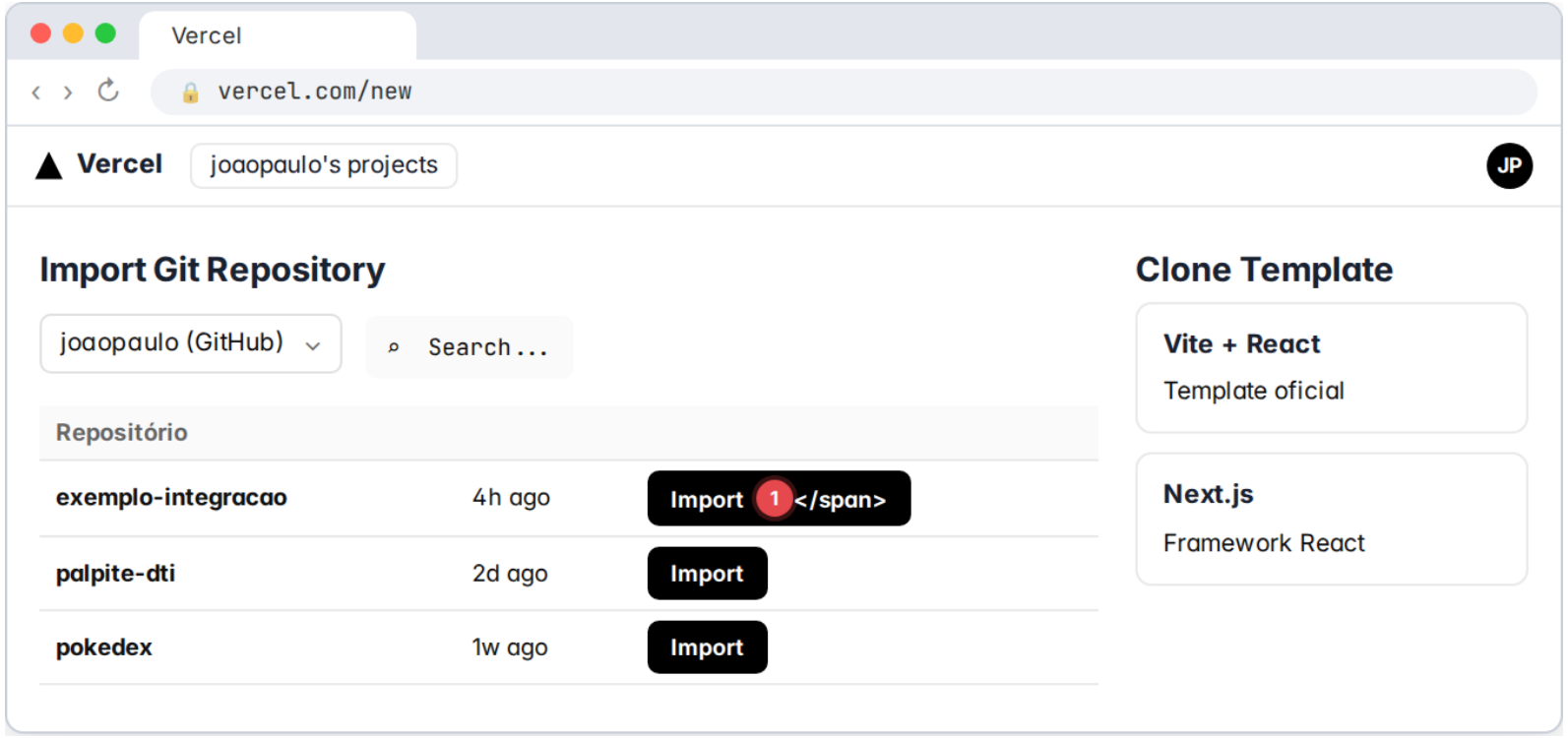
Aula 05 · Deploy

02

Passo 1: importar o repositório

Passo 1: importar o repositório

No painel do Vercel clique em **Add New… → Project**. Na primeira vez, autorize o Vercel a ler seus repositórios do GitHub (**Install** / **Adjust GitHub App Permissions**).

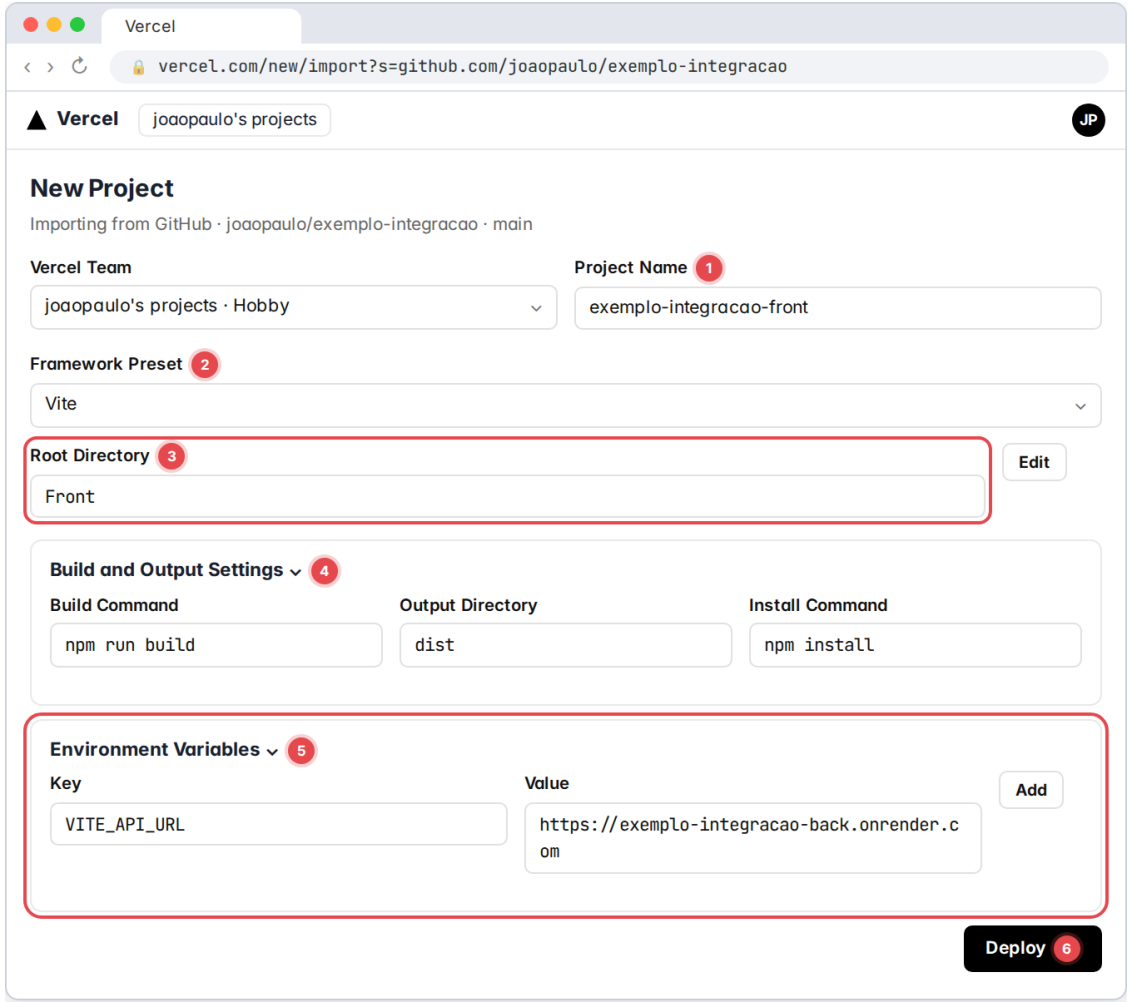


Tela **Import Git Repository**: escolha o repositório e clique em **Import**.

03

Passo 2: configurar o projeto

Passo 2: configurar o projeto



Configuração do projeto. O ponto crítico em monorepos é o **Root Directory**.

Passo 2: configurar o projeto

1. **Project Name**: vira a URL (`exemplo-integracao-front.vercel.app`). Use letras minúsculas e hífens.
2. **Framework Preset**: o Vercel detecta **Vite** sozinho ao ler o `package.json` da pasta.
3. **Root Directory**: clique em **Edit** e escolha a pasta **`Front`**. Sem isso, o Vercel procura o `package.json` na raiz e o build falha.
4. **Build and Output Settings**: os padrões do Vite já estão certos (`npm run build`, saída `dist`). Não altere.
5. **Environment Variables**: cadastre `VITE_API_URL` com a URL do back no Render (sem `/` no final). Se ainda não tem o back publicado, deixe para depois e faça redeploy (aula 07). Projetos com Supabase: `VITE_SUPABASE_URL` e `VITE_SUPABASE_PUBLISHABLE_KEY`.

Passo 2: configurar o projeto (cont.)

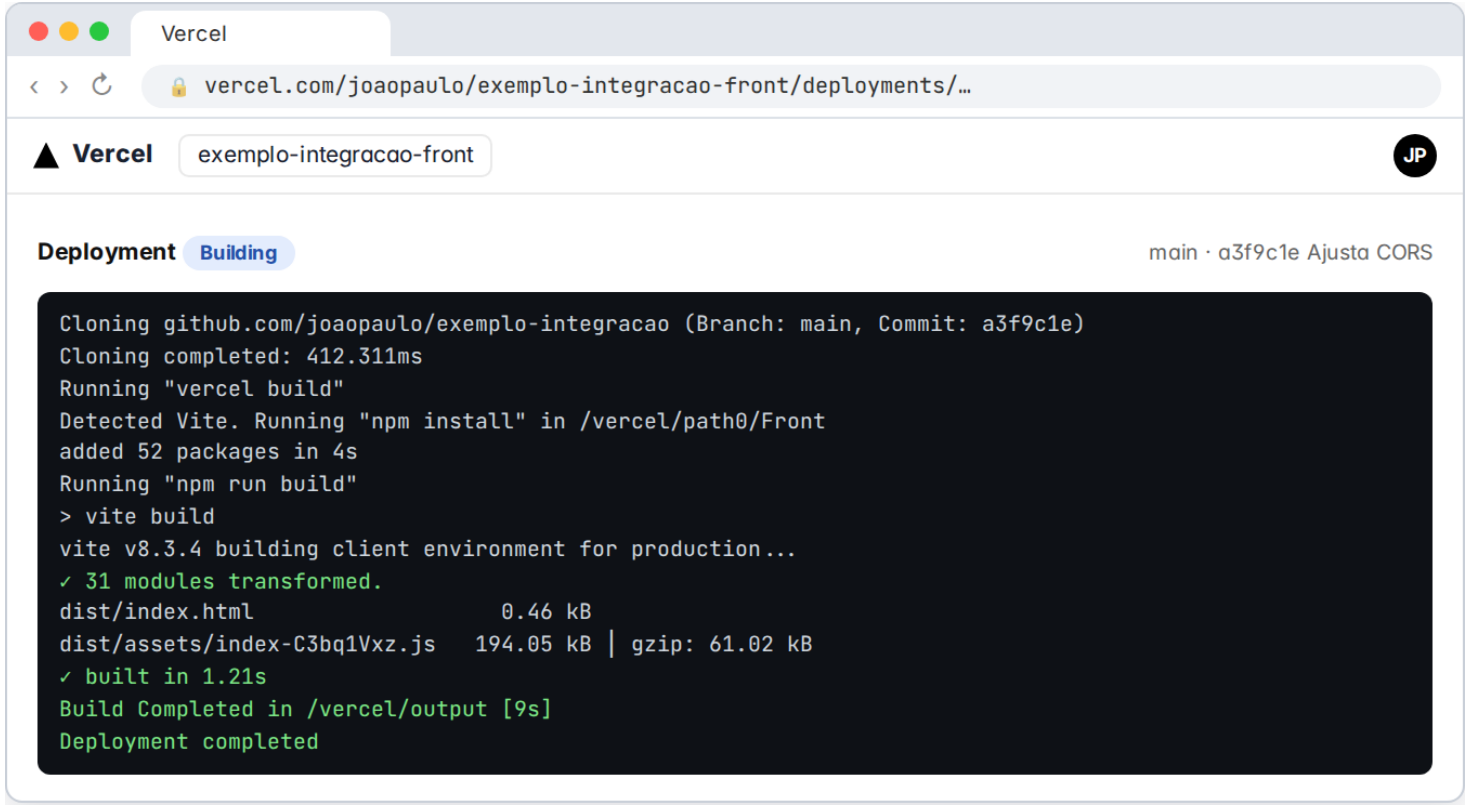
6

Clique em **Deploy**.

04

Passo 3: acompanhar o build

Passo 3: acompanhar o build

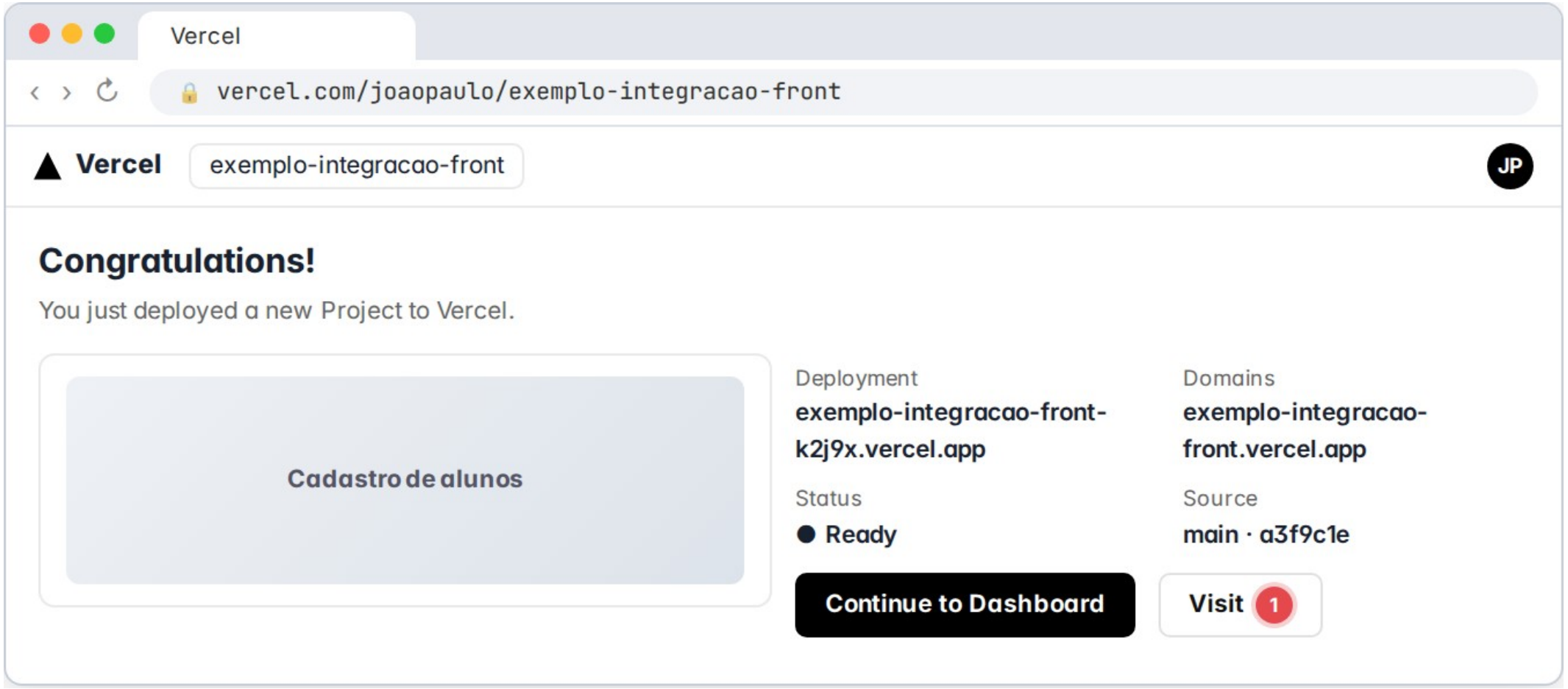


Log do build. Os mesmos comandos que você roda localmente (`npm install` e `npm run build`), dentro da pasta `Front`.

05

Passo 4: site no ar

Passo 4: site no ar



Deploy concluído. O domínio fixo `*.vercel.app` (Domains) é o que vai no `CORS_ORIGINS` do back.

Passo 4: site no ar

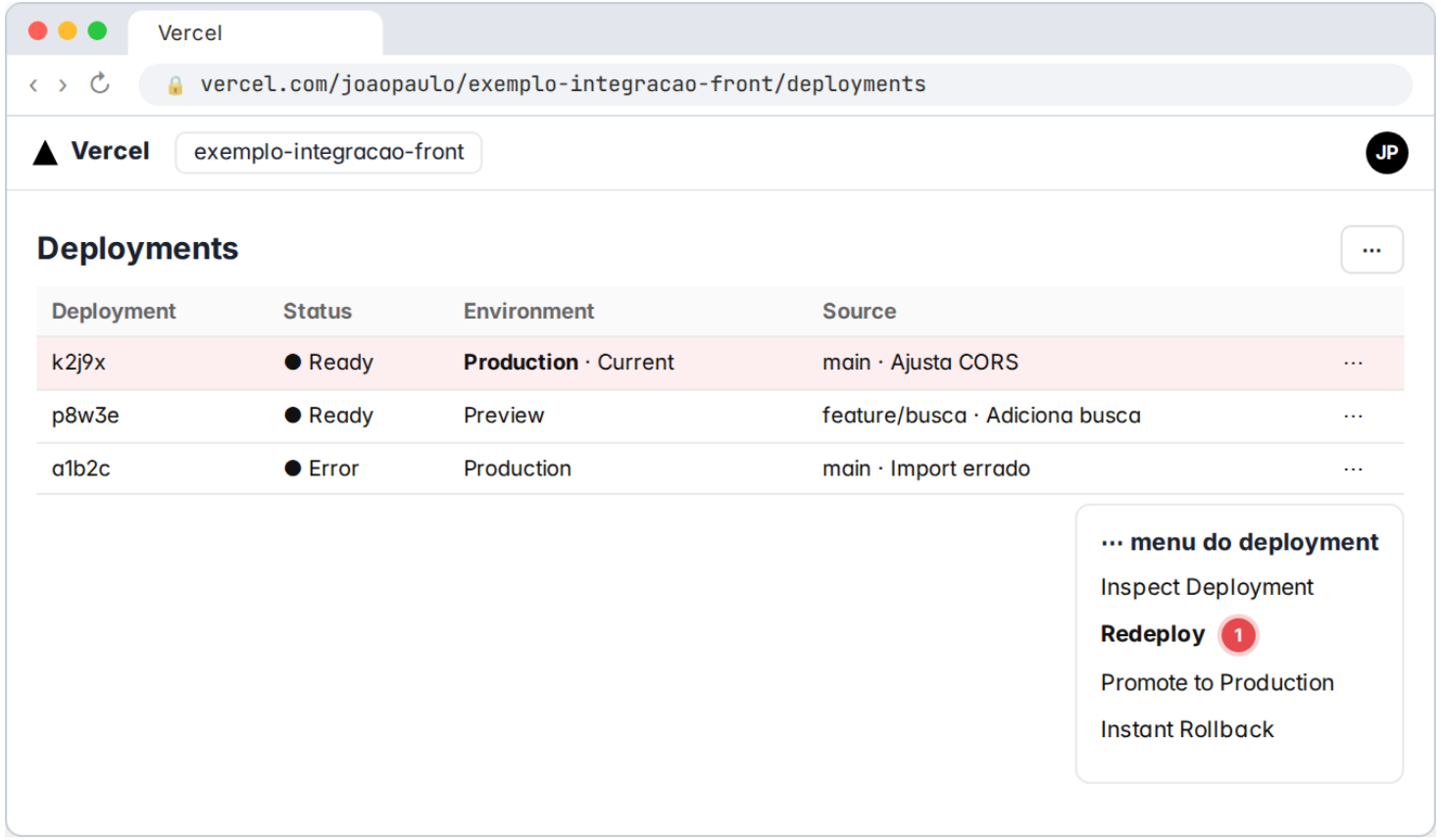
Observação

O Vercel mostra **dois tipos de URL**. O **domínio do projeto** (`exemplo-integracao-front.vercel.app`) sempre aponta para a última versão de produção: é essa que você divulga e coloca no CORS. A **URL do deployment** (`...-k2j9x.vercel.app`) é única de cada versão e nunca muda.

06

Passo 5: deploy contínuo e previews

Passo 5: deploy contínuo e previews



A partir de agora, todo `git push` na branch **main** gera um deploy de **produção**. Push em outras branches (ou *pull requests*) gera um **Preview Deployment** com URL própria, ótimo para testar antes de juntar na main.

Aba **Deployments**: histórico de versões, previews e o menu com **Redeploy** e **Instant Rollback**.

Passo 5: deploy contínuo e previews

Atenção

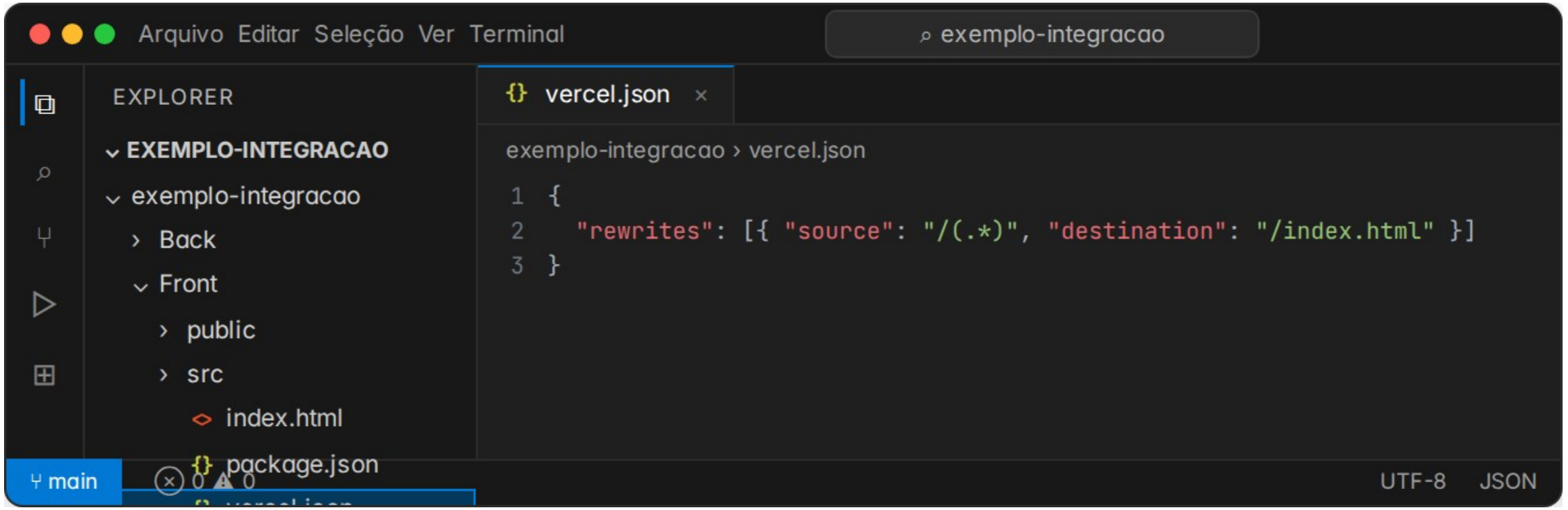
Mudou uma variável de ambiente? Faça Redeploy. O Vite grava `VITE_API_URL` no JavaScript **no momento do build**. Alterar a variável em *Settings* não muda o site que já está no ar: vá em **Deployments → ⋯ → Redeploy** (ou faça um novo push).

07

Passo 6: vercel.json para rotas de SPA

Passo 6: vercel.json para rotas de SPA

Com React Router, acessar `https://app.vercel.app/ranking` direto ou apertar F5 dá **404**: o Vercel procura um arquivo `ranking` que não existe. O *rewrite* manda tudo para o `index.html`:



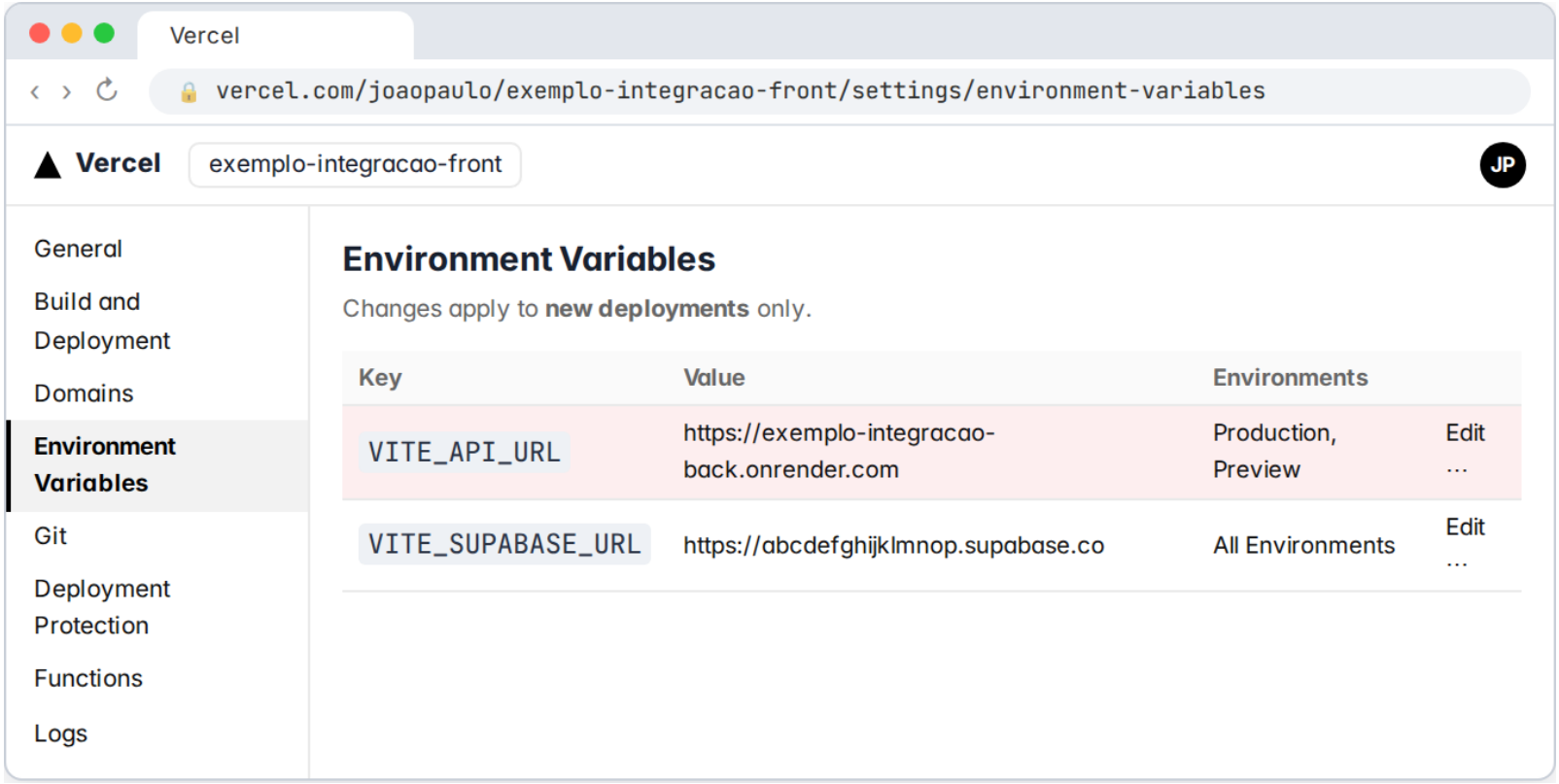
O `vercel.json` fica **dentro da pasta `Front`** (o Root Directory), igual aos projetos `AraCoffee` e `PalpiteDTI`.

08

Editando variáveis depois

Editando variáveis depois

Settings → Environment Variables. Cada variável pode valer para *Production*, *Preview* e *Development*. Depois de salvar, faça **Redeploy**.



Variáveis do projeto no Vercel.

09

Erros comuns no Vercel

Erros comuns no Vercel

Mensagem / sintoma	Causa	Solução
`Could not read package.json` / `No Output Directory named "dist"`	Root Directory errado	*Settings* → *Build and Deployment* → *Root Directory* = `Front`
`Could not resolve "./components/header"`	Maiúsculas/minúsculas no import (Linux diferencia)	O import deve bater exatamente com o nome do arquivo
Site abre, mas não carrega dados	`VITE_API_URL` ausente/errada, ou back dormindo	Confira a variável, faça Redeploy, acorde o Render
Erro de CORS no console do site publicado	URL do Vercel não está no `CORS_ORIGINS` do back	Aula 07
F5 em uma rota dá 404	Faltou o `vercel.json`	Passo 6
`Mixed Content` bloqueado	`VITE_API_URL` com `http://`	Use a URL `https://` do Render

10

Aprofundando: Vercel além do básico

Aprofundando: Vercel além do básico

Vercel CLI

```
npm install -g vercel
vercel login
cd Front
vercel            # deploy de preview, pergunta as configurações na primeira vez
vercel --prod     # deploy de produção
vercel env pull .env.local    # baixa as variáveis do projeto para o seu computador
```

Aprofundando: Vercel além do básico

vercel.json com cabeçalhos de cache e segurança

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }],
  "headers": [
    {
      "source": "/assets/(.*)",
      "headers": [{ "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }]
    },
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" }
      ]
    }
  ]
}
```

Rewrite x redirect

Rewrite

- A URL no navegador **não muda**
- O servidor entrega outro arquivo (index.html)
- Usado para SPA

vs

Redirect

- O navegador vai para **outra URL** (301/308)
- Ex.: `/antigo` → `/novo`
- Usado para mudar endereços

Aprofundando: Vercel além do básico

Configuração	Onde fica
Root Directory	Settings → Build and Deployment
Comando de build / pasta de saída	Settings → Build and Deployment
Versão do Node	Settings → Build and Deployment → Node.js Version
Variáveis	Settings → Environment Variables
Domínio próprio	Settings → Domains
Proteção de previews	Settings → Deployment Protection

Recursos úteis do Vercel

Preview por PR

Cada pull request ganha uma URL própria para revisão.

Instant Rollback

Volta à versão anterior em segundos.

Analytics

Visitas e desempenho real dos usuários (Web Vitals).

Ignored Build Step

Pula o build quando só a pasta `Back/` mudou.

Aprofundando: Vercel além do básico

Não refazer o front quando só o back mudou

```
# Settings → Build and Deployment → Ignored Build Step
git diff --quiet HEAD^ HEAD -- .
# sai com 0 (pula o build) se nada mudou dentro da pasta Front
```

Aprofundando: Vercel além do básico

Observação

O Vercel define `VERCEL_ENV` (`production`, `preview` ou `development`) e `VERCEL_URL`. Para usá-los no código Vite, crie variáveis com prefixo `VITE_` no painel que referenciem o valor desejado, já que o Vite só expõe `VITE_*`.

11

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Em um monorepo, qual configuração do Vercel é crítica?

A Node.js Version

B Root Directory = Front

C Domains

D Analytics

Resposta

Em um monorepo, qual configuração do Vercel é crítica?

A Node.js Version

✓ Root Directory = Front

C Domains

D Analytics

Por quê: Sem ela o Vercel não acha o package.json.

Quiz · questão 2 de 5

Pergunta

Mudou `VITE_API_URL` no painel. O site já usa a nova URL?

A Sim

B Não: precisa de Redeploy

C Só no preview

D Só no dia seguinte

Quiz · questão 2 de 5

Resposta

Mudou `VITE_API_URL` no painel. O site já usa a nova URL?

A. Sim

✓ Não: precisa de Redeploy

C. Só no preview

D. Só no dia seguinte

Por quê: O valor é gravado no build.

Quiz · questão 3 de 5

Pergunta

O que o vercel.json com rewrite resolve?

A CORS

B 404 ao dar F5 em rotas do React Router

C Build lento

D Variáveis

Quiz · questão 3 de 5

Resposta

O que o vercel.json com rewrite resolve?

A CORS

✓ 404 ao dar F5 em rotas do React Router

C Build lento

D Variáveis

Por quê: Toda rota cai no index.html.

Pergunta

Push em uma branch que não é a main gera:

A Nada

B Um Preview Deployment com URL própria

C Deploy de produção

D Erro

Resposta

Push em uma branch que não é a main gera:

A Nada

✓ Um Preview Deployment com URL própria

C Deploy de produção

D Erro

Por quê: Ótimo para revisar antes de mesclar.

Pergunta

Qual URL vai no CORS_ORIGINS do back?

A. A URL única do deployment

B. O domínio do projeto (*.vercel.app)

C. localhost

D. A URL do GitHub

Resposta

Qual URL vai no CORS_ORIGINS do back?

A. A URL única do deployment

✓ O domínio do projeto (*.vercel.app)

C. localhost

D. A URL do GitHub

Por quê: Ele sempre aponta para a produção atual.

Resumo da aula

- ✓ **Add New → Project → Import** o repositório; o Vercel detecta o Vite.
- ✓ Em monorepo, **Root Directory =** `Front`; build `npm run build`, saída `dist`.
- ✓ Cadastre `VITE_API_URL` (URL do Render, sem `/` final) e as chaves públicas do Supabase.
- ✓ Push na main = produção; outras branches = preview. Mudou variável: **Redeploy**.
- ✓ `vercel.json` com *rewrite* para `/index.html` resolve o 404 em rotas.

Exercícios

1. Publique o seu front no Vercel e envie a URL.

2. Crie uma branch, altere um texto, faça push e abra a URL de preview.

3. Remova o `vercel.json`, publique, reproduza o 404 no F5 e corrija.

4. Faça um deploy com erro de import proposital e use o **Instant Rollback**.

Para ir além

- vercel.com/docs/frameworks/vite
- vercel.com/docs/projects/environment-variables

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 05?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web